

Migrando o To-Do List de SQLite para Supabase

Este tutorial guia você na criação de um aplicativo To-Do (lista de tarefas) em Flutter utilizando o Supabase como backend, mantendo a arquitetura MVCS e adaptando a lógica da aplicação em SQLite que está em <https://github.com/madianitab/lista-de-tarefas-flutter-sqlite-mvcs> → copiar esse app (app_todolist_sqlite) e renomear para (app_todolist_supabase).

To-Do List SQLite vs. Supabase

- Como o projeto original tem uma separação clara de atribuições, usando o padrão MVCS, a Regra de Negócio e a Interface permanecem as mesmas, sendo necessário mudar apenas o "motor" de persistência.
- O que não precisa mudar:
 - A estrutura de pastas: o padrão MVCS é perfeito para essa migração, pois o impacto fica restrito à camada de dados. A estrutura não será alterada.
 - A View (**tarefa_page.dart**): não é necessário alterar, pois o `ListenableBuilder` e os `TextEditingController`s reagem ao `Controller`, independente de onde os dados vêm.
 - O `Controller` (**tarefa_controller.dart**): a lógica de estado (o `ChangeNotifier`, a lista de tarefas e o booleano carregando) permanece idêntica. Deve-se alterar apenas a chamada do serviço interno.
- O que precisa mudar:
 - O Modelo (**tarefa.dart**): no SQLite, a aplicação está convertendo booleanos para 0 ou 1. No Supabase (Postgres), o tipo `bool` é nativo. Deve-se remover as conversões `== 1` e trocar por valores booleanos reais.
 - O Serviço (**tarefa_service.dart**): no repositório original, ele instancia o `DatabaseService` (SQLite). Você deve alterar o conteúdo do arquivo atual para usar o cliente do Supabase (`Supabase.instance.client`).
 - Remoção do **database_service.dart** (SQLite): toda a parte de `openDatabase`, `getDatabasesPath` e os comandos SQL (`CREATE TABLE`, `INSERT INTO`) deixam de existir no código Flutter e passam a ser configurados diretamente no painel (Dashboard) do Supabase.
 - Configuração Inicial: será necessário adicionar o `Supabase.initialize` no `main.dart`, algo que não era necessário com o SQLite puro.

O que muda do SQLite para o Supabase

Recurso	SQLite (Local)	Supabase (Cloud)
Persistência	Arquivo local (.db)	Banco de dados remoto (Postgres)
Booleano	Tratado como 0 ou 1	Tratado como true ou false
Arquitetura	Requer <code>DatabaseService</code> manual	Usa SDK para chamadas HTTP/REST

Passo a passo das alterações na próxima página!!

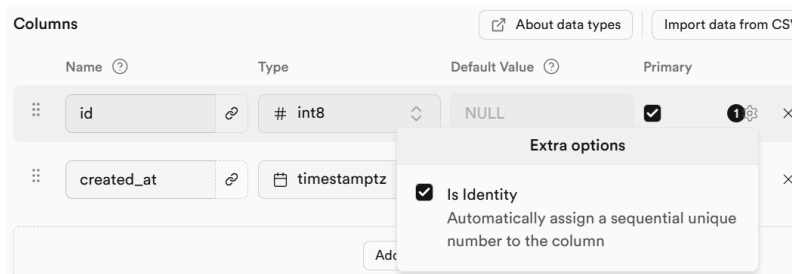
1. Cadastro e Configuração no Supabase

1. Acesse supabase.com e clique em "Start your project".
2. Faça login com sua conta GitHub.
3. Clique em "New Project" e selecione uma organização.
 - Se não tiver uma organização, vai em criar.
4. Dê um nome ao projeto (ex: *FlutterToDoList*), defina uma senha para o banco de dados e escolha a região mais próxima.
5. Mantenha o "Enable automatic RLS" desmarcado permitirá que você faça os primeiros testes de INSERT e SELECT no Flutter sem precisar configurar políticas de segurança complexas de imediato.
6. Aguarde alguns minutos enquanto o projeto é provisionado.

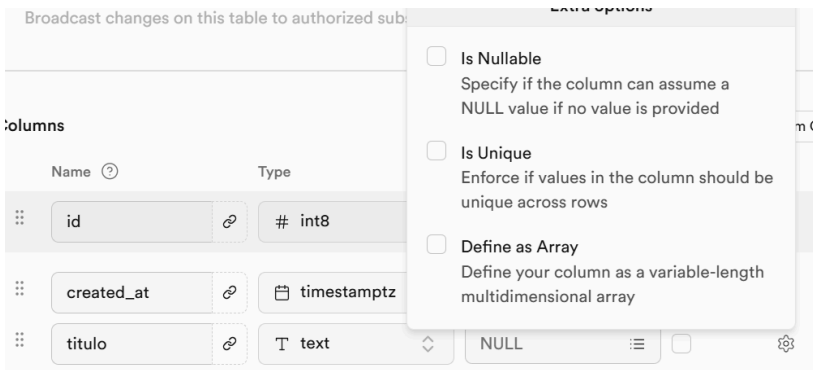
2. Criação da Tabela no Banco de Dados

No painel esquerdo do Supabase, vá em **Table Editor** -> **New Table**:

- **Name:** tarefas
- Desmarque a opção "Enable Row Level Security (RLS)"
- **Columns:**
 - id: int8, Primary Key, Autoincrement.



- titulo: text, Not Null.



- descricao: text, Not Null.
- concluida: bool, Default: false.

3. Configuração do Projeto Flutter

Execute os seguintes comandos:

```
flutter pub cache clean
flutter pub get
```

Acesse o arquivo **pubspec.yaml**:

Altere o nome da aplicação (name) para **app_todolist_supabase**.

Apague as dependências a seguir:

```
path: ^1.9.1
sqflite: ^2.4.2
sqflite_common_ffi: ^2.4.0
```

No terminal do seu projeto, adicione a dependência:

```
flutter pub add supabase_flutter
```

No seu arquivo main.dart, inicialize o Supabase antes de rodar o app:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await DatabaseService.inicializar();
  runApp(const MyApp());
}
```

4. Adaptação do Modelo (Model)

O modelo Tarefa muda levemente (linhas a serem alteradas estão em **negrito**) para lidar com o tipo booleano real do Postgres e o ID opcional:

```
class Tarefa {
  int? id;
  String titulo;
  String descricao;
  bool concluida;

  Tarefa({
    this.id,
    required this.titulo,
    required this.descricao,
    this.concluida = false
  });

  Map<String, dynamic> toMap() {
    return {
      if (id != null) 'id': id,
      'titulo': titulo,
      'descricao': descricao,
      'concluida': concluida,
    };
  }

  factory Tarefa.fromMap(Map<String, dynamic> map) {
    return Tarefa(
      id: map['id'],
      titulo: map['titulo'],
      descricao: map['descricao'],
      // Aceita tanto booleano nativo quanto o fallback antigo caso necessário
      concluida:mapa['concluida'] is bool ? mapa['concluida'] :
mapa['concluida']==1,
    );
  }
}
```

5. Criando o Supabase Service

5.1 - lib/services/tarefa_service.dart

O `TarefaService` agora vai herdar o papel de se comunicar diretamente com os dados, mas usando a sintaxe fluida e nativa do SDK do Supabase em vez de chamar métodos locais do SQLite.

Substitua o conteúdo do seu `tarefa_service.dart` pelo seguinte:

```
// lib/services/tarefa_service.dart

import 'package:supabase_flutter/supabase_flutter.dart';
import '../models/tarefa.dart';

class TarefaService {
  // Cliente HTTP pré-configurado que aponta para o servidor na nuvem
  final _client = Supabase.instance.client;

  // Busca todas as tarefas da tabela remota
  Future<List<Tarefa>> obterTarefas() async {
    final response = await _client
      .from('tarefas')
      .select()
      .order('id', ascending: true); // Ordena de forma crescente

    return response.map((item) => Tarefa.fromMap(item)).toList();
  }

  // Insere uma nova tarefa mapeando o objeto para JSON/Map
  Future<void> adicionarTarefa(String titulo, String descricao) async {
    final novaTarefa = Tarefa(titulo: titulo, descricao: descricao);
    await _client.from('tarefas').insert(novaTarefa.toMap());
  }

  // Altera o status booleano e envia a atualização filtrando pelo ID
  Future<void> alternarStatusTarefa(Tarefa tarefa) async {
    tarefa.concluida = !tarefa.concluida;
    await _client
      .from('tarefas')
      .update(tarefa.toMap())
      .eq('id', tarefa.id!); // Garante a atualização no registro
    // correto
  }

  // Remove o registro da tabela em nuvem através do ID correspondente
  Future<void> deletarTarefa(int id) async {
    await _client.from('tarefas').delete().eq('id', id);
  }
}
```

```
}
```

5.2 lib/services/tarefa_service.dart

O papel do seu DatabaseService agora passa a ser apenas um: estabelecer a conexão inicial com o servidor do Supabase.

A seguir, o código completo para substituir o conteúdo do arquivo **database_service.dart**, não esquecendo de acessar o **<https://supabase.com/>**, no bloco **Framework (Use a client library)** copiar a **URL** e no bloco **API Keys** copiar a chave (**anon public** em **Legacy anon, service_role API keys**):

```
import 'package:supabase_flutter/supabase_flutter.dart';

class DatabaseService {
  // Torna o construtor privado para evitar que a classe seja instanciada
  DatabaseService._();

  // Método estático responsável por inicializar a conexão com a nuvem
  static Future<void> inicializar() async {
    await Supabase.initialize(
      url: '**SUA_URL_DO_SUPABASE_AQUI**',
      anonKey: '**SUA_CHAVE_ANON_AQUI**',
    );
  }
}
```

EXERCÍCIO

Agora, insira a autenticação de usuários de duas formas:

1. **Tradicional (E-mail e Senha):** com obrigatoriedade de confirmação por e-mail antes do primeiro acesso.
2. **Login Social (Google):** utilizando o fluxo nativo do protocolo OAuth2.

